

LLM Cost Analysis — Preventify Diabetes Educator AI

Purpose: Token analysis, model selection rationale, and per-query cost estimates for all LLM calls in the pipeline.

Prepared: 2026-06-02

Models decided: Phase 1 → google/gemini-2.5-flash · Phase 2 → google/gemini-2.5-flash · Memory → google/gemini-2.0-flash-001

Routing: All calls via OpenRouter

1. LLM Call Map — Where Every Call Happens

Per patient query, the pipeline makes **2 LLM calls** in the normal path. A 3rd call fires as a background task once every 10 patient turns.

```
POST /chat
■■■ api/routes/chat.py:131 process_turn()
■■■ api/session_manager.py:232 handle_phase1()
■■■ engine/phase1.py:156 run_phase1() ← LLM CALL 1 (always)
■ ■■■ engine/phase1_runner.py:247
■
■■■ engine/phase1.py:223 run_phase2() ← LLM CALL 2 (conditional)
■■■ engine/phase2_runner.py:829

[Background – fires only at turn 10]
■■■ api/session_manager.py:264 end_session()
■■■ api/memory_compressor.py:77 compress_session() ← LLM CALL 3
```

Call 1 — Phase 1 Context Engine

- **File:** engine/phase1_runner.py:247
- **Fires:** Every single patient turn, no exceptions
- **Model param:** phase1_runner.py:65 — MODEL_ID
- **Max retries:** 2 attempts total (max_retries=1, loop is range(max_retries + 1))
- **Retry logic:** Only retries on timeout or rate_limit_429. All other errors (503, json_decode, api_error) break immediately after 1 attempt.
- **Output format:** Strict JSON (response_format={"type": "json_object"})
- **Settings:** max_completion_tokens=1024, temperature=0.1

Call 2 — Phase 2 RAG Pipeline

- **File:** engine/phase2_runner.py:829
- **Fires:** Conditionally — skipped when ANY of these are true:
 - context_sufficient = False (Phase 1 is asking a clarifying question)
 - intent == "escalation_only" (emergency — Risk Engine handles it)
 - risk_tier == 4 (Tier 4 emergency bypass)
- **Max retries:** 2 attempts total (same retry policy as Phase 1)
- **Output format:** Free-form prose

- **Settings:** `max_completion_tokens=1024, temperature=0.3`

Call 3 — Memory Compressor

- **File:** `api/memory_compressor.py:77`
- **Fires:** Background task, triggered at `session_manager.py:264` when `turn_number >= 10`
- **Frequency:** Once per session (every 10 patient turns), never per individual query
- **Patient is not waiting** — fully async, does not block the response
- **Output format:** Free-form prose (80–100 token clinical summary)
- **Settings:** `max_tokens=150, temperature=0.2`

Clarification Round — Maximum 1 Ask

When Phase 1 returns `context_sufficient=False`, Phase 2 is skipped and the bot sends a clarifying question. On the patient's next message, the loop guard at `engine/phase1.py:184` checks whether the last bot message ended with `?` — if yes, it forces `context_sufficient=True` regardless of what the model says.

Result: the system can ask for context at most 1 time. It is structurally impossible to ask a second clarifying question.

Turn N: Phase 1 LLM call → `context_sufficient=False` → clarifying question sent (Phase 2 skipped)
 Turn N+1: Phase 1 LLM call → loop guard fires → `context_sufficient` forced True → Phase 2 runs

2. Token Analysis

2.1 System Prompt Sizes (Fixed, Identical for All Users)

These are measured from the actual files on disk. The system uses `len(text) // 4` for token estimation (standard approximation for English prose).

Prompt	File	Characters	Tokens
Phase 1 system prompt	<code>prompts/phase1_system_prompt.txt</code>	18,325	4,581
Phase 2 system prompt	<code>prompts/phase2_system_prompt.txt</code>	16,963	4,240
Memory compressor prompt	inline in <code>memory_compressor.py:24</code>	288	72

The Phase 1 and Phase 2 system prompts are **static and identical across all users** — they never contain user-specific data. This is the key fact that makes prompt caching viable (see Section 4).

2.2 Phase 1 — Input Token Breakdown

Component	Min	Typical	Max	Notes
System prompt	4,581	4,581	4,581	Fixed — always the same
Session history (last 5 turns)	0	~300	~1,000	0 on first message; 5 turns × ~200 tok max

Component	Min	Typical	Max	Notes
Current patient message	3	~75	500	Validator caps at 2,000 chars = 500 tokens
Total input	4,584	~4,956	~6,081	

System prompt is **75% to 100%** of all Phase 1 input tokens.

2.3 Phase 1 — Output Token Breakdown

Output is a JSON object with 6 required fields. Size depends on whether clarifying questions are included.

Scenario	Tokens	Example
Simple QDS 1 — no clarification	~83	<code>{"intent":"general_dsmes", "qds_score":1, "context_sufficient":true, "clarifying_questions":[], ...}</code>
Typical QDS 3 — with profile signals	~130	Intent + full profile signals, no clarification
Max — 2 clarifying questions + full signals	~193	2 questions with button options, all signal fields populated
Hard cap	1,024	<code>max_completion_tokens</code> setting

Effective output range: 83–193 tokens. The 1,024 cap is never reached in practice.

2.4 Phase 2 — Input Token Breakdown

Phase 2 has a more complex input structure with several variable components.

Component	Min	Typical	Max	Notes
System prompt	4,240	4,240	4,240	Fixed
Phase 1 context block	0	~40	~90	Omitted when <code>intent=general_dsmes</code> and no location/family flags
Patient memory (<code>short_memory</code>)	0	~60	~100	0 for new users; ~100 tok compressed profile for returning users
Clinical chunks (top-5 RAG)	20	~1,250	~2,560	20 tok = "no evidence" fallback message; max = 5 chunks × 512 tok each
Placeholder assistant turn	25	25	25	Fixed string: "Understood. I have read the patient profile..."
Session history (last 5 turns)	0	~400	~700	5 turns × ~140 tok avg
Current patient message	3	~75	500	Same cap as Phase 1
Total input	4,288	~6,090	~8,200	

System prompt is **52% to 99%** of Phase 2 input tokens. In the typical case it is ~70%.

2.5 Phase 2 — Output Token Breakdown

The system prompt instructs 1–2 sentences for most responses, 3 sentences maximum for QDS 5 (distressed patients). Outputs are intentionally short — patients read on mobile in a chat window.

Scenario	Tokens	Notes
QDS 1 — 1 sentence general answer	~20–30	"HbA1c is your 3-month average blood sugar test."
QDS 2–3 — 2 sentences	~50–80	Typical for most queries
QDS 4–5 — up to 3 sentences	~80–150	Complication concern or distress
Real sample from logs (QDS 4 complex)	295	Constraint violation log — <code>phase2_failures.jsonl</code>
Hard cap	1,024	<code>max_completion_tokens</code> setting

Effective output range: 20–295 tokens. Average ~120–150 tokens.

2.6 Memory Compressor — Token Breakdown

Fires once per session (10 patient turns). Patient is not waiting.

Component	Min	Typical	Max	Notes
System prompt	72	72	72	Fixed, too small to cache
Full session transcript	35	~1,400	~6,500	10 patient turns + 10 bot turns
Total input	107	~1,472	~6,572	
Output	~50	~90	150	Hard cap: <code>max_tokens=150</code>

2.7 Total Per Query (Normal Path — Both Phases Run)

	Min	Typical	Max
Total input tokens	8,872	~11,046	~14,281
Total output tokens	~103	~250	~488

3. Prompt Caching — How It Works and Why It Matters

3.1 The Opportunity

Both system prompts are static and identical for every user. Right now they are paid at full input price on every single request. Since Phase 1 system prompt is 4,581 tokens and Phase 2 is 4,240 tokens — and these represent 70–100% of input tokens in each call — caching them is the single largest cost lever in the system.

3.2 How Caching Works Across All Users

The cache key is the **byte-identical prefix** of the request. Since the system prompt is the same for every user, any user's request warms the cache for every subsequent user.

User A sends first message of the day → cache MISS → full price paid, cache written
User B sends message 1 second later → cache HIT → 10% of normal price for system prompt
User C sends message 5 minutes later → cache HIT → 10% of normal price
...
10 minutes pass with no traffic → cache EXPIRES (Gemini TTL) or stays warm (Anthropic 5-min write, 1-hour write)
Next user after expiry → cache MISS → full price paid again, cache re-written

The cache is **provider-level and shared** — it is not per-user, per-session, or per-deployment. Any request to the same model with the same prefix is a cache hit.

3.3 Cache Discount by Provider (via OpenRouter)

Provider	Models	Cache Read Discount	Min Tokens	TTL
Google (Gemini)	2.5 Flash, 2.5 Pro, 2.0 Flash	90% off (pay 10% of input price)	4,096 tokens	Implicit automatic; ~10 min
Anthropic (Claude)	All Claude models	90% off (pay 10% of input price)	1,024 tokens	5-min TTL (1.25× write cost); 1-hour TTL (2× write cost)
OpenAI	GPT-4o, GPT-4o-mini	50–75% off	1,024 tokens	Automatic, no markup
DeepSeek	All DeepSeek models	50% off	Not specified	Automatic
Mistral / Llama	All	No caching	—	—

Phase 1 system prompt (4,581 tokens):

- Qualifies for Google caching (min: 4,096) ✓
- Qualifies for Anthropic caching (min: 1,024) ✓

Phase 2 system prompt (4,240 tokens):

- Qualifies for Google caching ✓
- Qualifies for Anthropic caching ✓

Memory compressor prompt (72 tokens): Does not qualify for any provider's minimum. Paid at full price — negligible.

3.4 Cache Savings on System Prompts

Without caching, system prompt tokens are paid at full input price every call. With caching, only the first call of each cache window pays full price.

At **10,000 calls/day** with cache warm for ~90% of traffic:

Call	Sys Prompt Tokens	Full Price/Call	Cached Price/Call	Daily Saving (90% hit rate)
Phase 1 (Gemini 2.5 Flash)	4,581	\$0.00137	\$0.000137	~\$11/day saved
Phase 2 (Claude Sonnet 4)	4,240	\$0.01272	\$0.001272	~\$102/day saved

At low traffic (< 1 req/10 min), cache is cold most of the time and savings are minimal. Caching pays off at volume.

4. Model Selection

4.1 Why These Models — The Core Logic

The use case does **not** need deep reasoning. Phase 1 is a classification task. Phase 2 is a grounded generation task (evidence is in the RAG chunks, not the model's parametric knowledge). Reasoning models (DeepSeek R1, o1, o3) add latency and cost with zero quality benefit here.

The two selection criteria in order of priority:

Reliability on the specific task — structured JSON output (Phase 1), strict safety rule following (Phase 2)

Cost efficiency — given the token volumes, caching discount is as important as base price

4.2 Phase 1 — Decided: google/gemini-2.5-flash

OpenRouter ID: google/gemini-2.5-flash

Price: \$0.30/M input · \$2.50/M output

Reason	Why It Matters for Phase 1
JSON schema strict mode	Supports <code>json_schema</code> with <code>strict: true</code> natively. Your Phase 1 schema has nested arrays (<code>clarifying_questions</code> , <code>profile_signals</code>). Strict mode guarantees schema adherence every call — no partial JSON, no missing fields.
90% cache discount on system prompt	4,581-token system prompt qualifies (Gemini min: 4,096). This is the biggest cost lever — system prompt is 75–100% of Phase 1 input.
1M context window	Phase 1 max input is 6,081 tokens. 994K headroom. Never a constraint.
~0.5s TTFT, 150+ tokens/sec	Phase 1 runs before Phase 2. Every ms here adds to total patient wait. Gemini 2.5 Flash is among the fastest models at this quality tier.
South Asian lay language	Google's training corpus includes substantial Indian English text. "White tablet", "sugar is high", "feet burning at night", "injection in the morning" — all map correctly to the clinical vocabulary in the schema.
Thinking disabled	Set <code>max_tokens_to_reasoning: 0</code> . Classification needs zero chain-of-thought. Disabling thinking removes latency and token overhead entirely.
Not chosen: DeepSeek R1	Reasoning model — 8–12s latency, reasoning tokens billed. Wrong tool.
Not chosen: Claude Sonnet 4	\$3/M vs \$0.30/M for an identical task. 10× price premium not justified for classification.
Not chosen: Gemini 2.5 Pro	30-second TTFT. Unacceptable for synchronous patient-facing Phase 1.

Budget fallback: google/gemini-2.0-flash-001 — \$0.10/M input, same JSON schema support, same 1M context, slightly older model. Use during internal testing.

4.3 Phase 2 — Decided: google/gemini-2.5-flash

OpenRouter ID: google/gemini-2.5-flash

Price: \$0.30/M input · \$2.50/M output

Same model as Phase 1. Decision is intentional — see reasoning below.

Reason	Why It Matters for Phase 2
90% cache discount on system prompt	4,240-token Phase 2 system prompt qualifies for Gemini's implicit caching (min: 4,096 tokens). Cached reads cost 10% of normal input price — the biggest single cost lever in Phase 2.
1M context window	Max Phase 2 input is 8,200 tokens. 991K headroom. No constraint whatsoever.
Strong safety rule following	Gemini 2.5 Flash reliably follows the Phase 2 safety rules ("never recommend doses", "never diagnose") from system prompt instruction. The <code>check_constraints()</code> function at <code>phase2_runner.py:855</code> provides a deterministic backstop regardless of model — making model-level safety a secondary concern.
Good South Asian cultural awareness	Google's training corpus includes substantial Indian English text. Handles "chaaya", "kappa", "matta rice", Kerala food context, and family-as-clinical-unit framing well.
RAG grounding	Phase 2 output is explicitly constrained by <code>chunk_usage_rules</code> in the system prompt: "do not add clinical facts not in the chunks." Gemini 2.5 Flash follows this reliably. Any failure is caught by <code>check_constraints()</code> before the patient sees it.
Consistent infra	Same model for Phase 1 and Phase 2 means one API key, one provider, one cache warm-up, one rate limit to manage. Operationally simpler.
Cost vs Claude Sonnet 4	Claude Sonnet 4 produces marginally warmer clinical prose. At \$3/M input vs \$0.30/M, the 10× price premium is not justified at this stage. Claude Sonnet 4 remains a future upgrade option if output quality becomes a clinical bottleneck after go-live.
Not chosen: Claude Sonnet 4	\$3.00/M input · \$15.00/M output — 10× more expensive. Quality advantage is real but not justified at current scale. Upgrade path after clinical sign-off if needed.
Not chosen: DeepSeek V3	Only 50% cache discount vs 90% for Gemini. Weaker on Kerala cultural context. Less proven on strict medical safety guardrails in production.
Not chosen: Llama 3.3 70B	No prompt caching on OpenRouter — eliminates the biggest cost saving.
Not chosen: Mistral Small 3	33K context window — eliminated. Phase 2 max input is 8,200 tokens, exceeds safe operating range with long session histories.

4.4 Memory Compressor — Decided: google/gemini-2.0-flash-001

OpenRouter ID: google/gemini-2.0-flash-001

Price: \$0.10/M input · \$0.40/M output

Patient is not waiting. This is a background batch task. Quality bar is lower (80–100 token clinical note). The 72-token system prompt does not qualify for caching at any provider. Cheapest adequate model with reliable instruction following.

4.5 Models Explicitly Rejected

Model	Price	Reason Rejected
deepseek/deepseek-r1	\$0.55/M in · \$2.19/M out	Reasoning model — 8–12s latency, reasoning tokens billed, zero benefit for classification or short-form prose
google/gemini-2.5-pro	\$1.25/M in · \$10/M out	30s TTFT — incompatible with synchronous patient-facing responses in both phases
mistralai/mistral-small-24b-instruct-2501	\$0.05/M in · \$0.08/M out	33K context window — eliminated from Phase 2
meta-llama/llama-3.3-70b-instruct	\$0.10/M in · \$0.32/M out	No prompt caching on OpenRouter; weaker clinical safety in production
openai/gpt-4o	\$2.50/M in · \$10/M out	Same cost tier as Claude Sonnet 4 but lower clinical warmth and safety reliability
qwen/qwen-2.5-72b-instruct	\$0.36/M in · \$0.40/M out	Partial JSON schema support; weaker Kerala cultural context; no clear advantage over Gemini 2.5 Flash

5. Final Cost Per Query

All costs calculated with **prompt caching active** and based on the token ranges established in Section 2.

5.1 Decided Models — Production Stack

Phase 1: google/gemini-2.5-flash (\$0.30/M input · \$2.50/M output)

Phase 2: google/gemini-2.5-flash (\$0.30/M input · \$2.50/M output)

Memory: google/gemini-2.0-flash-001 (\$0.10/M input · \$0.40/M output)

Both phases use the same model. Cache warm-up for Phase 1 also benefits Phase 2 at the provider level.

Phase 1 Cost Breakdown (per call)

Token Component	Tokens	Min	Typical	Max
System prompt — cached at ×0.10	4,581	\$0.000137	\$0.000137	\$0.000137
Variable input — uncached	3–1,500	\$0.000001	\$0.000113	\$0.000450
Output	83–193 tok	\$0.000208	\$0.000325	\$0.000483
Phase 1 total		\$0.000346	~\$0.000575	~\$0.001070

Phase 2 Cost Breakdown (per call)

Token Component	Tokens	Min	Typical	Max
System prompt — cached at ×0.10	4,240	\$0.000127	\$0.000127	\$0.000127
Variable input — uncached	48–3,960	\$0.000003	\$0.000552	\$0.001188
Output	20–295 tok	\$0.000050	\$0.000375	\$0.000738

Token Component	Tokens	Min	Typical	Max
Phase 2 total		~\$0.000180	~\$0.001054	~\$0.002053

Combined Per Query (Both Phases, Production Stack)

Scenario	Phase 1	Phase 2	Total
Minimum (new user, QDS 1, tiny message, no history)	\$0.000346	\$0.000180	\$0.000526
Typical (returning user, QDS 3, 3 prior turns, 5 avg chunks)	\$0.000575	\$0.001054	\$0.001629
Maximum (5 full turns, 5 large chunks, 2000-char message)	\$0.001070	\$0.002053	\$0.003123

5.3 Memory Compressor Cost (once per 10 turns)

Model: google/gemini-2.0-flash-001 (\$0.10/M input · \$0.40/M output)

Scenario	Input Tokens	Input Cost	Output Tokens	Output Cost	Total
Min (1-turn session)	107	\$0.0000107	~50	\$0.0000200	\$0.0000307
Typical (10 turns)	~1,472	\$0.0001472	~90	\$0.0000360	\$0.0001832
Max (10 turns, long msgs)	~6,572	\$0.0006572	150	\$0.0000600	\$0.0007172

Amortised over 10 turns: typical compressor cost adds **~\$0.000018 per query** — effectively zero.

5.4 Scale Projections (Production Stack — All Gemini 2.5 Flash)

Assumes 90% cache hit rate. Below ~500 queries/day the cache is cold more often — add 15–20% to estimates.

Daily Queries	Monthly Queries	Typical Cost/Query	Monthly Cost
100	3,000	\$0.001629	~\$5
500	15,000	\$0.001629	~\$24
1,000	30,000	\$0.001629	~\$49
5,000	150,000	\$0.001629	~\$244
10,000	300,000	\$0.001629	~\$489
50,000	1,500,000	\$0.001629	~\$2,444

5.5 Rollout Plan

Stage	Phase 1	Phase 2	Memory
Internal testing	Gemini 2.5 Flash	Gemini 2.5 Flash	Gemini 2.0 Flash

Stage	Phase 1	Phase 2	Memory
Clinical validation	Gemini 2.5 Flash	Gemini 2.5 Flash	Gemini 2.0 Flash
Production	Gemini 2.5 Flash	Gemini 2.5 Flash	Gemini 2.0 Flash
Future upgrade (if quality bottleneck found post go-live)	Gemini 2.5 Flash	Claude Sonnet 4	Gemini 2.0 Flash

6. Queries per \$1

Both phases always run. Cache warm (90% hit rate assumed).

Scenario	Cost/Query	Queries per \$1
Minimum (new user, QDS 1, no history, tiny message)	\$0.000526	~1,901
Typical (returning user, QDS 3, 3 prior turns, 5 avg chunks)	\$0.001629	~614
Maximum (5 full turns, 5 large chunks, 2000-char message)	\$0.003123	~320

The number to use for planning: ~614 queries per dollar (typical case).

At scale with warm cache: 1,000 real patient conversations costs roughly **\$1.63**.

7. Quick Reference

DECIDED STACK
██
Phase 1 google/gemini-2.5-flash \$0.30/M in · \$2.50/M out
Phase 2 google/gemini-2.5-flash \$0.30/M in · \$2.50/M out
Memory google/gemini-2.0-flash-001 \$0.10/M in · \$0.40/M out
Routing OpenRouter (all three calls)

TOKEN SIZES (measured from actual files)
██
Phase 1 system prompt 4,581 tokens (18,325 chars)
Phase 2 system prompt 4,240 tokens (16,963 chars)
Memory system prompt 72 tokens (288 chars)

PER QUERY COST (both phases, cache warm)
██
Phase 1 ~\$0.000575 [4,956 in / 130 out – typical]
Phase 2 ~\$0.001054 [6,090 in / 150 out – typical]
██
Total ~\$0.001629 per query (typical)

Min: \$0.000526 | Max: \$0.003123

QUERIES PER \$1
██
Minimum scenario ~1,901 queries
Typical ~ 614 queries ← use this for planning
Maximum scenario ~ 320 queries

12345678910111213141516171819202122232425262728293031323334353637383940414243444546474849505152535455565758596061626364656667686970717273747576777879808182838485868788899091929394959697989910010110210310410510610710810911011111211311411511611711811912012112212312412512612712812913013113213313413513613713813914014114214314414514614714814915015115215315415515615715815916016116216316416516616716816917017117217317417517617717817918018118218318418518618718818919019119219319419519619719819920020120220320420520620720820921021121221321421521621721821922022122222322422522622722822923023123223323423523623723823924024124224324424524624724824925025125225325425525625725825926026126226326426526626726826927027127227327427527627727827928028128228328428528628728828929029129229329429529629729829930030130230330430530630730830931031131231331431531631731831932032132232332432532632732832933033133233333433533633733833934034134234334434534634734834935035135235335435535635735835936036136236336436536636736836937037137237337437537637737837938038138238338438538638738838939039139239339439539639739839940040140240340440540640740840941041141241341441541641741841942042142242342442542642742842943043143243343443543643743843944044144244344444544644744844945045145245345445545645745845946046146246346446546646746846947047147247347447547647747847948048148248348448548648748848949049149249349449549649749849950050150250350450550650750850951051151251351451551651751851952052152

10,000 queries/day $\rightarrow$ ~\$489/month

50,000 queries/day $\rightarrow$ ~\$2,444/month